

1 — Recognize a Python, C++, or Ruby codebase

September 24, 2026 · Next: [Read source files](#)

This is a test-first implementation tutorial, not a description of functionality already shipped. Work from the repository root. Write each test, observe its failure, implement the smallest solution, then rerun the tests before refactoring. The complete solutions below let you compare your work after trying it yourself.

Start with what Reprobe already does

Read ARCHITECTURE.md, reprobe/commands.py, reprobe/model.py, and reprobe/cli.py. Reprobe already loads a local GGUF with --model, manages its lifetime, and runs interactive chat. GenerateChatReply demonstrates constructor injection and an execute() entrypoint. The repository argument is currently unused; no code review occurs. We will preserve that CLI while building separately testable review components. No network service, model download, or GPU is needed.

Install the project as described in README.md. For just these lightweight tests, a dependency-free development install is enough:

```
python -m pip install -e . --no-deps
python -m pip install pytest
python -m pytest -q
python -m reprobe --help
```

At the starting revision, pytest reports **17 passed and 2 failed**. The failures are ChatTests.test_cli_cleanup_on_all_chat_exits and ModelErrorTests.test_cli_reports_cleanup_failure. Both supply imaginary GGUF paths. They mock the backend and Path.is_file(), but the CLI now checks Path.exists() before reaching those mocks.

Before beginning new work, fix those fixtures: add patch.object(Path, "exists", return_value=True) to the with block **inside each of those two test methods only**. In the cleanup-on-all-exits test, add it to the existing block inside the loop; in the cleanup-failure test, add it alongside patch("reprobe.cli.chat",...). Both files already import Path and patch. Do not patch existence globally or change production validation. The tests/test_cli.py regression for a missing model must continue passing. Rerun python-mpytest-q; all 19 starting tests should now pass.

These fixture corrections are learner instructions; this documentation change does not apply them to the project.

Decide what “codebase” means

For this first version, a codebase is a directory containing at least one recognized source file, possibly nested. It need not contain .git. A README or pyproject.toml alone is insufficient. Recognition is deliberately a filename heuristic, not parsing or proof that code is valid.

Language	Recognized names
Python	.py, .pyw
C++	.cpp, .cc, .cxx, .h, .hpp, .hh, .hxx
Ruby	.rb, .rake, Rakefile, Gemfile, config.ru

Suffixes are matched without case sensitivity; the three special Ruby filenames are exact. Headers count as C++ candidates, although .h can also contain C. We do not claim C support. JavaScript, C, Go, and other languages are outside this version. A Gemfile counts because it is Ruby source, unlike a TOML manifest.

Skip symlink entries and these directory names anywhere in the tree: .git, .venv, venv, __pycache__, build, dist, vendor, .bundle. A root supplied through a symlink is resolved to its canonical directory; nested symlinks are not followed. These are explicit exclusions, not a .gitignore implementation.

The planned flow is:

```
InspectCodebase(path, repository).execute()
-> LocalRepository.resolve_root(path)
-> LocalRepository.discover(root)
<- CodebaseInspection(root, candidates, reason)
```

The command decides the workflow and classification. A repository component owns filesystem operations; immutable values carry results. Commands do not print, load models, or read environment settings.

Red: write the behavioral tests

Create tests/test_inspect_codebase.py:

```
from pathlib import Path
from unittest.mock import patch

import pytest

from reprobe.repository import LocalRepository, RepositoryError
from reprobe.review_commands import InspectCodebase
from reprobe.review_types import SourceCandidate

def inspect(path):
    return InspectCodebase(path, LocalRepository()).execute()

@pytest.mark.parametrize("name,language", [
    ("main.py", "python"), ("window.pyw", "python"),
    ("main.cpp", "cpp"), ("main.cc", "cpp"), ("main.cxx", "cpp"),
    ("api.h", "cpp"), ("api.hpp", "cpp"), ("api.hh", "cpp"),
    ("api.hxx", "cpp"), ("MAIN.CPP", "cpp"),
    ("main.rb", "ruby"), ("tasks.rake", "ruby"),
    ("Rakefile", "ruby"), ("Gemfile", "ruby"), ("config.ru", "ruby"),
])
def test_recognizes_supported_names_without_git(tmp_path, name, language):
    source = tmp_path / "src" / name
    source.parent.mkdir()
    source.write_text("example", encoding="utf-8")
    result = inspect(tmp_path)
    assert result.is_codebase
    assert result.root == tmp_path.resolve()
    assert result.candidates == (SourceCandidate(f"src/{name}", language),)
    assert result.languages == (language,)
    assert result.reason == "recognized_source"

def test_no_recognized_source_is_a_result_not_an_error(tmp_path):
    for name in ("README.md", "pyproject.toml", "app.js", "main.c", "main.go"):
        (tmp_path / name).write_text("example", encoding="utf-8")
    result = inspect(tmp_path)
    assert not result.is_codebase
    assert result.candidates == ()
    assert result.languages == ()
    assert result.reason == "no_supported_source"

def test_excluded_directories_and_sorted_results(tmp_path):
    for name in (".git", ".venv", "venv", "__pycache__", "build", "dist",
                 "vendor", ".bundle"):
        directory = tmp_path / "nested" / name
        directory.mkdir(parents=True)
        (directory / "hidden.py").write_text("pass", encoding="utf-8")
    for name in ("z.rb", "a.cpp", "m.py"):
        (tmp_path / name).write_text("example", encoding="utf-8")
    result = inspect(tmp_path)
    assert [c.path for c in result.candidates] == ["a.cpp", "m.py", "z.rb"]
    assert result.languages == ("cpp", "python", "ruby")

def test_nested_symlinks_are_not_followed(tmp_path):
    root = tmp_path / "repo"
    root.mkdir()
    external = tmp_path / "external"
    external.mkdir()
    (external / "secret.py").write_text("secret", encoding="utf-8")
    try:
```

```

        (root / "linked").symlink_to(external, target_is_directory=True)
        (root / "linked.py").symlink_to(external / "secret.py")
        (root / "loop").symlink_to(root, target_is_directory=True)
    except (OSError, NotImplementedError):
        pytest.skip("This environment cannot create symlinks")
    assert inspect(root).candidates == ()

def test_invalid_root_is_actionable(tmp_path):
    file = tmp_path / "main.py"
    file.write_text("pass", encoding="utf-8")
    for path in (tmp_path / "missing", file):
        with pytest.raises(RepositoryError, match="Repository"):
            inspect(path)

def test_filesystem_error_keeps_its_cause(tmp_path):
    failure = PermissionError("access denied")
    with patch("reprobe.repository.os.walk", side_effect=failure):
        with pytest.raises(RepositoryError, match="Cannot inspect") as raised:
            inspect(tmp_path)
    assert raised.value.__cause__ is failure

def test_command_accepts_a_repository_fake(tmp_path):
    class FakeRepository:
        def resolve_root(self, path):
            assert path == Path("input")
            return tmp_path

        def discover(self, root):
            assert root == tmp_path
            return (SourceCandidate("main.py", "python"),)

    result = InspectCodebase(Path("input"), FakeRepository()).execute()
    assert result.is_codebase
    assert result.root == tmp_path

```

Run:

```
python -m pytest tests/test_inspect_codebase.py -q
```

First you will get an import error because the new modules do not exist. Add the value types and filesystem component below, then create the command with only its constructor and an execute() that raises NotImplementedError. Rerun: tests must now fail when exercising missing behavior, not merely during import. Try implementing execute() yourself before comparing with the full solution.

Green: implement the smallest complete solution

Create reprobe/review_types.py. These types belong to the application, not to llama-cpp-python:

```

from dataclasses import dataclass
from pathlib import Path

@dataclass(frozen=True)
class SourceCandidate:
    path: str
    language: str

@dataclass(frozen=True)
class CodebaseInspection:
    root: Path
    candidates: tuple[SourceCandidate, ...]
    reason: str

@property

```

```

def is_codebase(self) -> bool:
    return bool(self.candidates)

@property
def languages(self) -> tuple[str, ...]:
    return tuple(sorted({item.language for item in self.candidates}))

```

Create reprobe/repository.py. Keep language recognition here so inspection and reading will share one policy. To add a language later, extend these rules and add behavioral tests; no registry or base class is necessary.

```

import os
from pathlib import Path

from reprobe.review_types import SourceCandidate

LANGUAGE_SUFFIXES = {
    "python": frozenset({".py", ".pyw"}),
    "cpp": frozenset({".cpp", ".cc", ".cxx", ".h", ".hpp", ".hh", ".hxx"}),
    "ruby": frozenset({".rb", ".rake"}),
}
RUBY_FILENAMES = frozenset({"Rakefile", "Gemfile", "config.ru"})
EXCLUDED_DIRECTORIES = frozenset({
    ".git", ".venv", "venv", "__pycache__", "build", "dist", "vendor", ".bundle",
})

class RepositoryError(RuntimeError):
    """Repository access failed; not a negative codebase classification."""

def detect_language(path: Path) -> str | None:
    if path.name in RUBY_FILENAMES:
        return "ruby"
    suffix = path.suffix.lower()
    for language, suffixes in LANGUAGE_SUFFIXES.items():
        if suffix in suffixes:
            return language
    return None

def raise_walk_error(error: OSError) -> None:
    # os.walk otherwise ignores directory enumeration errors by default.
    raise error

class LocalRepository:
    def resolve_root(self, path: Path) -> Path:
        try:
            root = path.expanduser().resolve(strict=True)
            if not root.is_dir():
                raise RepositoryError(f"Repository must be a directory: {root}")
            return root
        except (OSError, RuntimeError) as exc:
            if isinstance(exc, RepositoryError):
                raise
            raise RepositoryError(f"Repository cannot be accessed: {path}: {exc}") from exc

    def discover(self, root: Path) -> tuple[SourceCandidate, ...]:
        candidates = []
        try:
            for directory, dirs, files in os.walk(
                root, topdown=True, followlinks=False, onerror=raise_walk_error
            ):
                parent = Path(directory)
                dirs[:] = sorted(
                    name for name in dirs
                    if name not in EXCLUDED_DIRECTORIES

```

```

        and not (parent / name).is_symlink()
    )
    for name in files:
        path = parent / name
        if path.is_symlink() or not path.is_file():
            continue
        language = detect_language(path)
        if language is not None:
            candidates.append(SourceCandidate(
                path.relative_to(root).as_posix(), language
            ))
    except OSError as exc:
        raise RepositoryError(f"Cannot inspect repository {root}: {exc}") from exc
    return tuple(sorted(candidates, key=lambda candidate: candidate.path))

```

Create `reprobe/review_commands.py`. Keep the existing `chat` command intact.

```

from pathlib import Path

from reprobe.repository import LocalRepository
from reprobe.review_types import CodebaseInspection

class InspectCodebase:
    def __init__(self, path: Path, repository: LocalRepository) -> None:
        self._path = path
        self._repository = repository

    def execute(self) -> CodebaseInspection:
        root = self._repository.resolve_root(self._path)
        candidates = self._repository.discover(root)
        reason = "recognized_source" if candidates else "no_supported_source"
        return CodebaseInspection(root, candidates, reason)

```

The annotation documents the concrete collaborator; Python still allows the small fake used in the test. We have no need for another protocol yet. The result's properties derive language and classification information instead of storing potentially contradictory fields.

Validate, review, and refine

```

python -m pytest tests/test_inspect_codebase.py -q
python -m pytest -q
python -m reprobe --help

```

Review the ownership boundary: the command classifies; `LocalRepository` performs I/O; values describe the result; only a caller decides how to display it. Do not change CLI behavior just to call the new command. When you implement this lesson, update `ARCHITECTURE.md` to describe these implemented components while keeping LLM review in the planned section. After any refinement, rerun the tests.

This implementation assumes a local tree that is not being maliciously modified while inspected. It does not parse syntax, apply `.gitignore`, discover every possible language filename, or establish a race-proof filesystem sandbox. These are explicit boundaries, not evidence that unrecognized directories have no code.